

Recent Advances in Code Reasoning and Generation with Large Language Models: A Survey

InternLM*¹

* indicate equal contribution

Abstract

The rapid advancements in large language models (LLMs) have significantly impacted the fields of code reasoning and generation. In this survey, we provide a comprehensive overview of recent developments that leverage LLMs to improve reasoning capabilities through code generation and execution. We categorize the research into key themes, including mathematical and numerical reasoning, program generation and execution, structured reasoning, and interactive and autonomous systems. We also discuss the challenges and limitations faced by current approaches, as well as their implications for future research directions. This survey aims to provide a detailed understanding of how LLMs are transforming code reasoning and generation, paving the way for future innovations in this domain.

1 Introduction

In recent years, the advent of large pre-trained language models (LLMs) such as GPT-3 (?) and Codex (?) has revolutionized the landscape of code reasoning and generation. These models, with their ability to generate and reason about code, have enabled new possibilities in automating complex tasks that were traditionally handled by human programmers. The integration of code with natural language processing (NLP) capabilities in these models has sparked a paradigm shift, allowing for

the seamless blending of code execution and language understanding.

Traditional methods of code generation and reasoning often relied on explicit rules or domain-specific knowledge, limiting their applicability to diverse tasks. However, the emergence of LLMs has introduced a more generalized approach, where the models learn to perform reasoning tasks by generating and executing code. This shift has opened up new avenues for research, particularly in areas that require complex reasoning, such as mathematical problem solving, structured data manipulation, and interactive system development.

The purpose of this survey is to provide a detailed overview of the current state of research on code reasoning and generation using LLMs. We categorize the research into several key areas, including mathematical and numerical reasoning via code, program generation and execution for enhanced reasoning, leveraging code for structured reasoning, and the development of interactive and autonomous systems. Additionally, we examine the challenges associated with these approaches, such as model interpretability, efficiency, and the handling of edge cases, which are critical for the deployment of these models in real-world applications.

As LLMs continue to evolve, their potential to transform code reasoning and generation becomes

increasingly evident. This survey not only highlights the advancements made so far but also identifies the limitations and open challenges that need to be addressed to fully realize the capabilities of these models. We conclude by suggesting directions for future research that could further enhance the integration of LLMs in code reasoning and generation tasks, ultimately leading to more robust and efficient systems.

2 Program-Aided Language Models for Reasoning

2.1 Mathematical and Numerical Reasoning via Code

DPO’s differentiation lies in its ability to integrate human feedback directly for policy optimization, which simplifies the training process by eliminating the need for a learned reward function. This approach has been further enhanced by incorporating diverse divergence constraints, as demonstrated by Wang et al. (2024) (Wang et al., 2024). Similarly, SLiC-HF (Zhao et al., 2023) addresses preference optimization by producing similar objectives to DPO. Despite these advancements, integrating LLMs with programming capabilities for mathematical reasoning tasks remains a challenge.

GPT-4 Code Interpreter excels at solving complex math problems by generating and executing code, evaluating execution results, and self-correcting when necessary. This capability is bolstered by a novel method called Code-based Self-Verification (CSV), where the model verifies its answers using code and corrects them if they are erroneous (Kang et al., 2023).

Recent developments in datasets and fine-tuning methods for code-assisted LLMs have further improved their mathematical reasoning performance. MathCoder models, fine-tuned on datasets such as MATH and GSM8K, have significantly outperformed earlier models by generating code-based solutions for math problems (Liu et al., 2024). Moreover, MuMath-Code combines tool-use LLMs with Python interpreters and data augmentation, achieving state-of-the-art performance on mathematical reasoning benchmarks (Zhang et al., 2022).

Although DPO and related approaches demonstrate potential, future research should explore methods to effectively leverage code assistance and self-correction in LLMs to tackle even more complex mathematical problems.

2.2 Program Generation and Execution for Enhanced Reasoning

Chain of Code (CoC) is a novel approach that has been introduced to improve the reasoning abilities of language models (LLMs) by leveraging code execution and emulation. The key innovation in CoC is the ability of LLMs to format semantic sub-tasks into flexible pseudocode that can be captured by an interpreter, which then hands off undefined behaviors to an LLM-based code emulator for simulation. This capability broadens the scope of problems that LLMs can address, especially those that require a mix of logic, arithmetic, and semantic reasoning. The effectiveness of CoC is evident from its performance in various benchmark tests, where it consistently outperforms other baselines, demonstrating improved accuracy and expanded reasoning capabilities (Li et al., 2023).

Another significant contribution comes from the Think-and-Execute framework, which breaks down the reasoning process into two distinct steps. Think and act. In the Think phase, an LLM discovers a task-level logic shared across all instances of a given task and expresses it in pseudocode. The Execute phase involves tailoring the pseudocode to individual instances and simulating its execution. This framework effectively guides LLMs towards discovering task-level logic that can be reused across different instances, enhancing their reasoning abilities without the need for instance-specific codes. The experiments carried out on seven algorithmic reasoning tasks underscore the superiority of this method over traditional instance-specific reasoning approaches, showcasing the potential of pseudocode to better steer the reasoning of LLMs (Chae et al., 2024).

ReGAL, or Refactoring for Generalizable Abstraction Learning, is another innovative framework that addresses the limitations of LLMs in generating reusable code. By refactoring existing programs without altering their execution output, ReGAL identifies and learns generalizable abstractions that can be utilized across diverse domains. This method improves the accuracy of LLMs in predicting programs, as illustrated by their performance on various datasets, including LOGO, Date reasoning, TextCraft, and MATH. Notably, ReGAL-augmented LLMs outperform the baseline models across multiple domains, underscoring the efficacy of learning reusable functions through refactoring (Stengel-Eskin et al., 2024).

In sum, these methodologies, CoC, Think and Execute, and ReGAL, represent significant strides in enhancing the coding and reasoning abilities of LLMs. Through innovative approaches to code generation, execution, and refactoring, these frameworks pave the way for more efficient, accurate, and generalizable problem-solving in the realm of programming and algorithmic reasoning. As research in this area continues to evolve, we can foresee even greater advancements in the capabilities of LLMs to tackle complex programming tasks.

3 Code Integration and Structural Reasoning

3.1 Leveraging Code for Structured Reasoning and Generalization

In contrast, the Hybrid Question Answering (HQA) approach, exemplified by HProPro, tackles the challenge of reasoning over diverse data sources by leveraging program-based prompting and code execution. This method outperforms baseline systems in few-shot settings, as it does not require specialized retrievers or modal transformations (?). The effectiveness of executable code actions in enhancing LLM capabilities is evident.

The multilingual structured reasoning dataset xSTREET highlights a performance gap between English and non-English reasoning tasks in LLMs. To bridge this gap, augmenting a code dataset with multilingual comments and employing step-by-step code primitives during inference significantly improves performance in xSTREET tasks, without compromising general-purpose abilities (Li et al.).

The PyBench evaluation framework presents a comprehensive testbed for LLM agents on various real-world coding tasks, demanding complex reasoning and code execution over multiple turns. This benchmark underscores the necessity for robust understanding of Python packages and superior reasoning capabilities (Zhang et al.). The fine-tuned PyLlama3 model exhibits superior performance, outpacing larger models, underscoring the importance of executable code actions for complex coding tasks.

The CoSm prompting method enhances LLMs' code simulation capabilities, reducing reliance on pattern recognition and memorization (Malfa et al.). CodeMind evaluates LLM reasoning abilities across multiple tasks and reveals notable performance drops with increased complexity (Liu et al.). Lastly, executing natural language algo-

rithms through LLMs has shown promise, particularly for lightweight tasks, highlighting the potential for executable code actions in natural language understanding (Zheng et al.).

In summary, the integration of executable code actions represents a crucial advancement in LLM capabilities, offering enhanced performance and the ability to tackle complex, multilingual reasoning tasks effectively. This methodological approach not only bridges existing performance gaps, but also opens new avenues for LLM applications across diverse computational domains.

3.2 Evaluation and Challenges in Code Reasoning with LLMs

In a similar vein, Ukai et al. (2024) tackle the challenge of long input prompts in visual programmatic models (VPM) by introducing AdaCoder, an adaptive prompt compression framework. AdaCoder operates in two phases: a compression phase that generates compressed pre-prompts based on specific question types and an inference phase where the appropriate pre-prompt is selected to generate code. This framework significantly reduces token length by 71.1

Similarly, Shen et al. (2024) introduce PyramidCoder, a hierarchical code generator for compositional Visual Question Answering (VQA). PyramidCoder improves VQA accuracy by leveraging a single frozen LLM and pre-defined prompts across three hierarchical levels, demonstrating its flexibility and state-of-the-art performance on various VQA datasets.

On the topic of code simulation, Chen et al. (2024b) explores the novel task of logic code simulation, where LLMs emulate logical solvers to predict the results of logical programs. Their introduced Dual Chains of Logic (DCoL) technique enhances LLM performance by advocating a dual-path thinking approach, achieving a notable improvement in accuracy with GPT-4-Turbo.

Furthermore, Chen et al. (2024a) address the limitations of existing code reasoning benchmarks by proposing REval, a framework for evaluating code reasoning abilities and consistency with program execution. Their large-scale empirical study reveals the urgent need to strengthen the code reasoning capability of code LLMs, reflecting unsatisfactory performance across both Runtime Behavior Reasoning and Incremental Consistency Evaluation.

Lastly, Xue et al. (2024) introduce SelfPiCo, a

framework that dynamically guides the execution of partial code by incorporating an open-source LLM within an interactive loop. SelfPiCo uses few-shot in-context learning and chain-of-thought reasoning to continuously learn and refine predictions, outperforming existing tools, and showcasing its practical usage and potential applications in software debugging and testing.

4 Interactive and Autonomous Code Generation

4.1 Interactive and Multi-Agent Code Generation Systems

Another notable development is the ChatDev framework, designed by [Qian et al. \(2023\)](#), which integrates LLM-driven specialized agents to facilitate coordinated and communicative software development across the design, coding, and testing phases. This unified language-based approach enhances collaboration and reduces technical inconsistencies, illustrating the potential of linguistic communication in enabling multiagent task-solving.

The MetaGPT framework proposed by [Hong et al. \(2023\)](#) further advances this collaboration by encoding standardized operating procedures into prompt sequences, reducing errors and enhancing workflow efficiency. This innovative meta-programming approach allows agents to verify intermediate results, thereby generating more coherent solutions for complex problems.

The CodeChain framework, introduced by [Le et al. \(2023\)](#), addresses the challenge of generating modularized code by iteratively refining and reusing submodules. This method significantly improves both modularity and solution correctness, achieving 35

Furthermore, the CodeAgent framework by [Zhang et al. \(2024\)](#) integrates external programming tools to aid LLMs in real-world repo-level tasks, enhancing performance by up to 250

In summary, these advances in LLM-based collaborative frameworks highlight significant strides made in automated code generation. Using multi-agent systems, unified communication, and modular approaches, these frameworks not only improve code quality, but also extend the capabilities of LLMs in tackling complex and real-world coding tasks.

4.2 Autonomous Code Generation and Self-Improvement

Large language models (LLMs) have shown promising capabilities in code generation tasks, yet they still face challenges, particularly in terms of code quality, reliability, and integration into broader software development workflows. To address these issues, various frameworks and methodologies propose to leverage LLMs in conjunction with standard software engineering practices. This section provides a comprehensive review of cutting-edge frameworks such as [Lin et al. \(2024\)](#), [Bouzenia et al. \(2024\)](#), and [Tao et al. \(2024\)](#), showcasing their contributions to improving the reliability and acceptability of LLM-generated code within software development environments.

[Lin et al. \(2024\)](#) introduce the LCG (LLM-based Code Generation) framework, which simulates established software development processes via multiple LLM agents. By assigning specific roles such as requirement engineer, architect, developer, tester, and scrum master to LLMs, this approach mirrors distinct phases of software development and fosters collaboration. The agents iteratively refine themselves through chain-of-thought and prompt composition techniques, producing higher-quality code. Compared to baseline models like GPT, LCGScrum notably enhances code generation performance across benchmarks like HumanEval and MBPP, demonstrating an average improvement of +15%. This underscores the importance of integrating software development practices into LLM training, emphasizing how structured processes can significantly elevate code quality.

Building on this, [Bouzenia et al. \(2024\)](#) presents the RepairAgent, an autonomous LLM-based framework for program repair. Unlike conventional deep learning methods, RepairAgent enables LLMs to autonomously plan and execute repair actions by using a suite of specialized tools. This flexibility allows the agent to dynamically assess bugs, gather repair ingredients, and validate fixes. Evaluated on the Defects4J dataset, RepairAgent independently repairs 39 bugs unresolved by prior techniques, highlighting the efficacy of integrating autonomous decision-making capabilities into program repair workflows.

To further bridge the gap between LLM and practical software development, [Tao et al. \(2024\)](#) propose MAGIS, a novel LLM-based multi-Agent framework tailored to resolve GitHub issues.

Through custom-designed agents such as Manager, Repository Custodian, Developer, and Quality Assurance Engineer, MAGIS orchestrates collaborative efforts in planning and coding. This framework not only substantially improves issue resolution rates but also enhances LLMs’ practical applicability in complex software development environments.

The frameworks reviewed collectively illustrate the potential of integrating LLMs with structured software engineering practices. By emulating human-like collaborative development processes, these approaches enable LLMs to generate not only syntactically correct code but also code that aligns with broader software development standards and practices. This integration marks a significant step towards realizing the full potential of LLMs in real-world software development scenarios.

Lin et al. (2024)’s LCG framework successfully integrates LLMs into software development processes, enhancing code quality through collaborative refinement. In contrast, Bouzenia et al. (2024)’s RepairAgent framework extends the application of LLMs in software engineering by enabling autonomous program repair. The MAGIS framework proposed by Tao et al. (2024) further bridges the gap between LLMs and practical software development by customizing multi-agent collaborations for GitHub issue resolution. Each framework contributes uniquely to the integration of LLMs into software development, showcasing the potential of structured practices to enhance LLM-generated code.

5 Refinement, Optimization, and Benchmarking in Code Generation

5.1 Iterative Refinement and Self-Optimization

One significant method developed is Self-Refine, introduced by Madaan et al. (2023). It leverages the iterative feedback mechanism that LLMs use to refine text, applying this to code generation. Through this approach, an LLM generates an initial code snippet which then serves as the basis for subsequent refinement stages, where feedback from the same LLM is used to improve the code. In particular, this process does not require supervised training data, additional training, or reinforcement learning, utilizing the LLM only for generation, feedback, and refinement. Evaluation across various tasks shows this method significantly enhances the ini-

tial code quality, improving task performance by approximately 20

Another method, Self-Debugging, as proposed by Chen et al. (2023), teaches LLMs to debug their own code. This self-learning process involves demonstrating the model with examples of debugging scenarios and allowing it to identify and correct errors by analyzing execution results. It excels in improving code generation performance on complex programming tasks and notably enhances the model’s sample efficiency by reusing failed predictions.

The Self-Edit approach, devised by Zhang et al. (2023a), incorporates the results of the execution into the code generation process. By executing the generated code and using the results to guide editing, it effectively corrects errors, especially on competitive programming tasks. It enhances the pass@1 rate by 89

In addition to refining the concept of self-correction, the CYCLE framework, introduced by Ding et al. (2024), focuses on improving the self-refinement capability of LLM. It leverages feedback from execution results to improve faulty generations, leading to a consistent improvement in code quality, especially in scenarios where initial code predictions are incorrect.

These self-optimization methods collectively demonstrate the potential for LLMs to enhance their own code generation capabilities through iterative refinement and self-correction. Although each method presents unique advantages and scenarios where it performs best, they collectively signal a promising direction in the development of more autonomous and self-improving code generation models.

5.2 Benchmarking and Evaluation of LLM-Based Code Generation

CodeNav’s architecture is characterized by its dynamic indexing and searching of code snippets within the target repository, followed by their progressive import and execution. This iterative refinement culminates in solutions that are meticulously aligned with user queries. The framework’s uniqueness lies in its capability to handle complex coding queries through three primary modules: a Code Graph Generator, a Code Snippet Indexer, and an Execution Feedback Evaluator. These modules collectively facilitate a systematic, step-by-step solution generation process, enhancing the precision

and applicability of AI-generated code.

In comparison to other state-of-the-art models like AgileCoder, which adopts an agile methodology to divide software development into distinct roles and phases (Nguyen et al., 2024), CodeNav simplifies the workflow by focusing on iterative code enhancement through real-time execution feedback. This streamlines the development process and ensures a continuous alignment of the generated code with user intent.

Benchmarking results underscore the remarkable efficacy of CodeNav. Compared to tool-use LLM agents, which require manual tool registrations, CodeNav’s dynamic code retrieval and iterative execution lead to significantly more accurate and contextually relevant solutions. This superior performance is further demonstrated in comparative evaluations on diverse codebases, where CodeNav consistently outperforms tool use approaches by capturing nuances in real-world code repositories that are inaccessible to manually registered tools.

CodeNav’s success highlights the potential of integrating real-world code repositories for iterative solution refinement, underscoring the viability of an iterative, execution feedback-driven coding paradigm. This framework not only showcases the transformative shift in LLM-based coding but also lays the groundwork for future advancements in interactive, AI-assisted software development.

5.3 Advanced Techniques in Code Repair and Generation

The advent of Imitation Learning from Language Feedback (ILF) further revolutionizes code generation by incorporating natural language feedback during training, rather than relying solely on demonstrations (?). This approach not only reduces the feedback burden on users but also enhances model performance, as indicated by the 38% increase in pass@1 accuracy on Mostly Basic Python Problems (MBPP) (?). This finding underscores the efficacy of integrating natural language feedback into the training process.

Reflecting on these advancements, the introduction of text-driven feedback in the form of error messages and stack traces for code generation tasks (LErTi) stands as a pioneering innovation (Wang et al., 2023). By iteratively fine-tuning models on concatenated instructions, generated programs, and textual feedback, LErTi achieves state-of-the-art performance on MBPP and generalizes well to

other datasets (Wang et al., 2023). This method demonstrates the potential of textual feedback in enhancing both the quality of the generation and the efficiency of the sample.

Moreover, the exploration of explora-exploit trade-offs in refinement strategies has informed the development of more effective iterative refinement algorithms (Tang et al., 2024). This work leverages Thompson Sampling to balance the exploitation of well-performing programs with the exploration of less-tested alternatives, resulting in improved problem-solving capabilities.

In summary, the integration of interactive generation, feedback refinement, and natural language feedback into program synthesis methodologies represents a significant advance in code generation. These innovations not only enhance model performance but also pave the way for more efficient and effective code generation tasks.

6 Conclusion

In this paper, we provide a comprehensive survey of recent advances in code reasoning and generation facilitated by large-language models (LLMs). We categorized the research into several key areas, including mathematical and numerical reasoning, program generation and execution, structured reasoning, and the development of interactive and autonomous systems. For each category, we have discussed representative works, highlighting their contributions, challenges, and the potential impact on future research.

Our survey underscores the transformative role that LLMs play in enhancing code reasoning capabilities, offering new paradigms that go beyond traditional rule-based and domain-specific approaches. The integration of code generation with natural language processing opens up vast possibilities for automating complex reasoning tasks, yet it also brings forth significant challenges, such as the need for better model interpretability, efficiency, and robustness.

We believe that the insights provided in this survey will serve as a valuable resource for researchers and practitioners in the field, offering a clear understanding of the current landscape and guiding future endeavors. As LLMs continue to evolve, it is crucial to address the existing limitations and explore new avenues that will further enhance the application of these models in code reasoning and generation. We anticipate that continued explo-

ration and development in this domain will lead to even more sophisticated and capable systems, ultimately pushing the boundaries of what is possible in automated reasoning and programming.

References

- Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. [Repairagent: An autonomous, llm-based agent for program repair](#).
- Hyungjoo Chae, Yeonghyeon Kim, Seungone Kim, Kai Tzu-iunn Ong, Beong-woo Kwak, Moohyeon Kim, Seonghwan Kim, Taeyoon Kwon, Jiwan Chung, Youngjae Yu, and Jinyoung Yeo. 2024. [Language models as compilers: Simulating pseudocode execution improves algorithmic reasoning in language models](#).
- Junkai Chen, Zhiyuan Pan, Xing Hu, Zhenhao Li, Ge Li, and Xin Xia. 2024a. [Reasoning runtime behavior of a program with llm: How far are we?](#)
- Minyu Chen, Guoqiang Li, Ling-I Wu, Ruibang Liu, Yuxin Su, Xi Chang, and Jianxin Xue. 2024b. [Can language models pretend solvers? logic code simulation with llms](#).
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. [Teaching large language models to self-debug](#).
- Yangruibo Ding, Marcus J. Min, Gail Kaiser, and Baishakhi Ray. 2024. [Cycle: Learning to self-refine the code generation](#).
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2023. [Metagpt: Meta programming for a multi-agent collaborative framework](#).
- Hong Kang, Yuandu Shen, Peng Li, and Shuang Ren. 2023. [Reward-agnostic and continuous human feedback for llm via preference and program synthesis](#). *arXiv:2303.14562*.
- Hung Le, Hailin Chen, Amrita Saha, Akash Gokul, Doyen Sahoo, and Shafiq Joty. 2023. [Codechain: Towards modular code generation through chain of self-revisions with representative sub-modules](#).
- Bryan Li, Tamer Alkhoul, Daniele Bonadiman, Nikolaos Pappas, and Saab Mansour. 2024. [Eliciting better multilingual structured reasoning from llms through code](#). In *Proceedings of the Web Conference (WSDM)*.
- Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei Fei, Fei Xia, and Brian Ichter. 2023. [Chain of code: Reasoning with a language model-augmented code emulator](#).
- Feng Lin, Dong Jae Kim, and Tse-Husn Chen. 2024. [When llm-based code generation meets the software development process](#).
- Changshu Liu, Shizhuo Dylan Zhang, Ali Reza Ibrahimzade, and Reyhaneh Jabbarvand. 2020. [Code-mind: A framework to challenge large language models for code reasoning](#). In *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2020)*.
- Pengfei Liu, Ruocheng Wu, Yuechen Wu, and Jiaqi Su. 2024. [Chain-of-thought reasoning for large-scale language models](#). *arXiv:2404.01678*.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. [Self-refine: Iterative refinement with self-feedback](#).
- Emanuele La Malfa, Christoph Weinhuber, Orazio Torre, Fangru Lin, Samuele Marro, Anthony Cohn, Nigel Shadbolt, and Michael Wooldridge. 2024. [Code simulation challenges for large language models](#). *arXiv: Computing Research Repository*.
- Minh Huynh Nguyen, Thang Phan Chau, Phong X. Nguyen, and Nghi D. Q. Bui. 2024. [Agilecoder: Dynamic collaborative agents for software development based on agile methodology](#).
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. [Chatdev: Communicative agents for software development](#).
- Ruoyue Shen, Nakamasa Inoue, and Koichi Shinoda. 2024. [Pyramid coder: Hierarchical code generator for compositional visual question answering](#).
- Elias Stengel-Eskin, Archiki Prasad, and Mohit Bansal. 2024. [Regal: Refactoring programs to discover generalizable abstractions](#).
- Hao Tang, Keya Hu, Jin Peng Zhou, Sicheng Zhong, Wei-Long Zheng, Xujie Si, and Kevin Ellis. 2024. [Code repair with llms gives an exploration-exploitation tradeoff](#). In *Proceedings of the 34th IEEE International Conference on Computer-Design (ICCD)*.
- Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. 2024. [Magis: Llm-based multi-agent framework for github issue resolution](#).
- Mahiro Ukai, Shuhei Kurita, Atsushi Hashimoto, Yoshitaka Ushiku, and Nakamasa Inoue. 2024. [Adacoder: Adaptive prompt compression for programmatic visual question answering](#).

- Tianhe Wang, Taifeng Lei, Kaicheng Yu, Wenqin Jin, Xi Zhou, and Yu Qi. 2024. [Reverse engineering preference-aware models by adversarial training to generate diverse descriptions](#). *arXiv:2404.08521*.
- Xingyao Wang, Hao Peng, Reyhaneh Jabbarvand, and Heng Ji. 2023. [Leti: Learning to generate from textual interactions](#). In *Proceedings of the 39th International Conference on Computational Linguistics (COLING)*.
- Zhipeng Xue, Zhipeng Gao, Shaohua Wang, Xing Hu, Xin Xia, and Shanping Li. 2024. [Selfpico: Self-guided partial code execution with llms](#).
- Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. [Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges](#).
- Kechi Zhang, Zhuo Li, Jia Li, Ge Li, and Zhi Jin. 2023a. [Self-edit: Fault-aware code editor for code generation](#).
- Wenhao Zhang, Jinwei Shao, Qinghong Yang, Lei Chen, Fei Huang, Jianfeng Guo, Rongrong Ni, Guangxu Wang, Jiajia Qun, Kexuan Zhang, Jin Wang, Tong Zhang, Hong Liu, Yuyu Zhang, Fei Yao, Yanbo Zuo, Hongxiao Yao, and Ting Wang. 2022. [Program-aided language models](#). *arXiv:2211.10435*.
- Yaolun Zhang, Yinxu Pan, Yudong Wang, and Jie Cai. 2023b. [Pybench: Evaluating llm agent on various real-world coding tasks](#). In *Proceedings of the 30th International Flairs Conference on Artificial Intelligence (FLAIRS-23)*.
- Yuqing Zhao, Xueqing Liu, Hong Kang, and Zheng-Hua Yang. 2023. [Slic-hf: Preference-aware large language model via high-level instruction following](#). *arXiv:2308.07104*.
- Xin Zheng, Qiming Zhu, Hongyu Lin, Yaojie Lu, Xi-anpei Han, and Le Sun. 2024. [Executing natural language-described algorithms with large language models: An investigation](#). In *Proceedings of the 2024 International Conference on Web Information Systems Engineering (WISE)*.